Capítulo 1

Introducción al lenguaje Pascal: concepto de algoritmo y programa, acciones = sentencias ejecutables, estructura de un programa Pascal, concepto de tipos de datos, asignación, entrada/salida

1. Algoritmo

El diseño e implementación de **algoritmos** constituye una metodología de resolución de problemas muy usada en programación de computadoras.

Se denomina **algoritmo** a un procedimiento con las siguientes características:

1. Poseer una secuencia finita y ordenada de acciones primitivas (órdenes elementales)
2. Las acciones primitivas son ***no*** ambiguas
3. La ejecución de la secuencia de acciones primitivas, en el orden dado, conduce a la solución del problema en un tiempo finito

Las **acciones** son las tareas o trabajos que realiza el procesador. Estas acciones pueden ser:

a) Primitivas: su enunciado es suficiente para que el procesador pueda ejecutarlo sin información adicional.
b) No primitivas: son órdenes no elementales de una abstracción mayor a las órdenes elementales. Cada una de ellas puede ser dividida en otras acciones (primitivas o no).

Una vez que el algoritmo ha sido correctamente diseñado e implementado en algún lenguaje apropiado, puede ser usado para solucionar problemas específicos. Para obtener la solución, se debe recurrir a una entidad capaz de ejecutar el algoritmo. Esa entidad es denominada, en un sentido amplio, **procesador.** Entonces, un procesador es toda entidad capaz de "entender" órdenes elementales y llevarlas a cabo.

Los algoritmos deben ser expresados en un lenguaje, por ejemplo la lengua natural (para nosotros el español).

Ejemplos:

E0) Se tiene un procesador que realiza las siguientes acciones primitivas: leer (ingresar o captar) datos numéricos- almacenar datos numéricos -sumar dos datos – multiplicar dos datos. Se desea resolver el siguiente problema: calcular el valor de una función $f(x)=ax+b$ para un x_0 dado.

ALGORITMO RECTA

1) Leer y almacenar los valores de a, b, y x_0.
2) Multiplicar a por x_0 y almacenarlo
3) Sumar b al valor obtenido en 2)
4) Fin del ALGORITMO RECTA

E1) Problema: encontrar la raíz positiva de la siguiente ecuación cuadrática $x^2-4x+4=0$
Un algoritmo para este problema puede ser enunciado con las siguientes acciones y constituye una solución:

ALGORITMO RAIZ1

1) Ingresar los valores de los coeficientes de la ecuación
2) Calcular el discriminante
3) Calcular las raíces
4) Fin de algoritmo RAIZ1

A fin de identificar los algoritmos, estos poseen un encabezamiento donde, además de figurar la palabra ALGORITMO, se escribe un nombre. Luego de esta cabecera, se enumeran las acciones en orden. Finalmente, se indica la terminación del algoritmo con la acción 4. Sin embargo, el lenguaje con el que se expresó el algoritmo RAIZ1 debe abandonar la lengua natural (con cierta ambigüedad) para tomar el formalismo (mas rígido) de los lenguajes programación. Por ejemplo, expresemos el mismo algoritmo usando un lenguaje más formal (sin ser un lenguaje de programación):

Solución:

ALGORITMO RAIZUNO

1) Ingresar los valores 1, -4 y 4
2) **Discriminante** $\leftarrow (-4)^2 - 4*1*4$
3) **Raiz** $\leftarrow \dfrac{4+\sqrt{\text{Discriminante}}}{2*1}$
4) Fin del algoritmo RAIZUNO

La acción primitiva de ingresar indica que el algoritmo necesita cierta información para trabajar apropiadamente. En el presente ejemplo esa información (o *datos*, como veremos posteriormente) son los coeficientes de la ecuación. El símbolo ← indica una acción primitiva de **reemplazo** del valor actual de la variable que figura a la izquierda de ← por el resultado del cálculo de la expresión que figura a la derecha. Las palabras que figuran en negrita representan variables (en forma similar a las variables matemáticas).

E2) Problema: encontrar las raíces de la siguiente ecuación cuadrática $x^2-4x+3=0$

Solución:

ALGORITMO RAIZDOS

1) Ingresar los valores 1, -4 y 3
2) **Discriminante** $\leftarrow (-4)^2 - 4*1*3$
3) **Raiz1** $\leftarrow \dfrac{4+\sqrt{\text{Discriminante}}}{2*1}$
4) **Raiz2** $\leftarrow \dfrac{4-\sqrt{\text{Discriminante}}}{2*1}$

5) Fin del algoritmo RAIZDOS

Si bien se han resuelto los problemas planteados, los algoritmos usados carecen de una importante cualidad que es la ***generalidad.*** Los algoritmos RAIZUNO y RAIZDOS solucionaron problemas muy particulares. Mas adelante se planteará un algoritmo para tratar la búsqueda de las raíces de una ecuación cuadrática ***cualquiera*** $a x^2 + b x + c = 0$

E3) Problema: Dados 3 números enteros (cualesquiera), se desea detectar los que son positivos y escribirlos. Vamos a usar una nueva acción primitiva que nos permita tomar una decisión.

Solución:

ALGORITMO DECISION_1

1) Ingresar **a,b,c**
2) Si **a** es positivo entonces escribir **a**
3) Si **b** es positivo entonces escribir **b**
4) Si **c** es positivo entonces escribir **c**

5) Fin del algoritmo DECISIÓN_1

Este algoritmo acepta tres números enteros y escribe los que son positivos. A fin de precisar las acciones primitivas 2,3 y 4 las escribimos de la siguiente manera:

ALGORITMO DECISION_2
1) Ingresar **a,b,c**
2) Si **a>0** entonces escribir **a**
3) Si **b>0** entonces escribir **b**
4) Si **c>0** entonces escribir **c**
5) Fin del algoritmo DECISIÓN_2

La decisión que se toma en 2,3 y 4 establece que se escribe el número entero solo si es mayor que cero, no se establece acción si el número es menor o igual que cero.

E4) Problema: Dados 3 números enteros (cualesquiera), se desea escribir aquellos que sean positivos y aquellos que no lo sean, aclarando cada caso. Vamos a usar una nueva acción primitiva que nos permita tomar una decisión más compleja.
Solución:

ALGORITMO DECISION_3
1) Ingresar **a,b,c**
2) Si **a>0** entonces escribir **a,**'es positivo'
sino escribir **a**,'es negativo o cero'
3) Si **b>0** entonces escribir **b,**'es positivo'
sino escribir **b**,'es negativo o cero'
4) Si **c>0** entonces escribir **c,**'es positivo'
sino escribir **c**,'es negativo o cero'
5) Fin del algoritmo DECISIÓN_3

Vemos que la decisión discrimina entre valores mayores que cero (la acción que se sigue es escribir que el número es positivo) de aquellos que no lo son (la acción que se sigue es escribir que es negativo o cero).

E5) Problema: Dada una *cantidad variable* de números enteros (cualesquiera), se desea escribir aquellos que sean positivos y aquellos que no lo sean, aclarando cada caso.

El algoritmo presenta el siguiente desafío: si la cantidad variable es 3 (tres) el algoritmo es DECISION_3, pero si la cantidad es 50, el algoritmo se torna altamente ineficiente porque deberíamos repetir 50 veces acciones primitivas como 2. Por lo tanto, debemos recurrir a una nueva acción primitiva: la que nos permite repetir otras acciones sin necesidad de escribirlas tantas veces como se la necesite.

ALGORITMO REPETICION
1) Ingresar cantidad
2) Repetir las dos siguientes acciones (desde que la variable contador toma el valor 1 hasta cantidad)
   3) Ingresar **a**
   4) Si **a>0** entonces escribir **a,**'es positivo'
   sino escribir **a**,'es negativo o cero'
5) Terminar la repetición
6) Fin del algoritmo REPETICION

Considerando que los algoritmos son métodos para solucionar problemas, veamos **como trabaja** uno de ellos para alcanzar tal fin. A continuación presentaremos un ejemplo de resolución de un problema específico usando el algoritmo REPETICION. El procesador que trabaja con el algoritmo es un ser humano adulto que entiende correctamente las acciones y puede ejecutarlas. Los datos de los que se dispone son: **cantidad** es 4, y los 4 números enteros son: 6,-9, 0, 2

Resolución del problema (=ejecución del algoritmo):

ENTRADA: cantidad ← 4	{Ingresa 4 en cantidad }
contador ←1	{contador indica que se va a procesar el primer número}
a ← 6	{Ingresa 6 en a}
6 es positivo	{Compara 6 > 0 y escribe 6 es positivo}
contador ←2	{contador indica que se va a procesar el segundo número}
a ← -9	{Ingresa -9 en a}
-9 es negativo o cero	{Compara -9>0 y escribe -9 es negativo o cero}
contador ←3	{contador indica que se va a procesar el tercer número}
a ← 0	{Ingresa 0 en a}
0 es negativo o cero	{Compara 0>0 y escribe 0 es negativo o cero}
contador ←4	{contador indica que se va a procesar el cuarto número}
a ← 2	{Ingresa 2 en a}
2 es positivo	{Compara 2 > 0 y escribe 2 es positivo}

Notar que, en cada repetición, hemos usado la misma variable **a** para almacenar cada uno de los números enteros. En cada nuevo reemplazo, se pierde el valor anterior.
Entre las llaves {} se han escrito comentarios aclaratorios para el lector.

La ejecución de un algoritmo es una secuencia operaciones calculadas a partir de las acciones primitivas del algoritmo.
La ejecución de un algoritmo para una entrada específica de datos constituye la resolución de un problema particular. Esos datos son propios del problema.

E6) Problema: Escribir un algoritmo para el método de Euclides: encontrar el máximo común divisor de dos enteros positivos **dividendo** y **divisor** (esto es, el entero positivo mas grande **dcm** que divide a ambos).
En este problema, no tratamos de encontrar el divisor común mayor de dos números particulares como 45 y 3; se desea encontrar el divisor común mayor para cualquier par de dos enteros positivos, a fin de obtener un algoritmo **general**.

Veamos un primer algoritmo:

ALGORITMO EUCLIDES
- **1)** Ingresar los valores correspondientes a las variables **dividendo** y **divisor**
- **2)** Calcular el **cociente** (división entera) entre **dividendo** y **divisor**
- **3)** Calcular el **resto**
- **4)** Repetir las siguientes acciones (mientras el **resto** no es 0)
  - **5)** reemplazar el valor de **dividendo** por **divisor** y el de **divisor** por **resto**,
  - **6)** calcular nuevamente **cociente** y **resto**
- **7)** Terminar repetición
- **8)** Escribir **divisor** {es el MCD}
- **9)** Fin del algoritmo EUCLIDES

Más específicamente (y sin ambigüedad):

ALGORITMO EUCLIDES

1) Ingresar **dividendo** y **divisor**
2) **cociente ←dividendo ÷ divisor**
3) **resto ← dividendo – cociente * divisor**
4) Repetir las siguientes acciones (mientras resto <> 0)
   5) **dividendo ← divisor**
   6) **divisor ← resto**
   7) **cociente ← dividendo ÷ divisor**
   8) **resto ← dividendo – cociente * divisor**
9) Fin repetir
10) **mcd ← divisor** {este es el caso cuando resto=0}
11) Escribir **mcd**
12) Fin del algoritmo EUCLIDES

El símbolo ÷ indica división entera (15/2=7.5 pero 15÷2=7). Las acciones primitivas de este algoritmo son más complejas que las de los algoritmos anteriores. Las acciones en 1, 2 y 3 ya fueron vistas, pero la 4 implica una repetición de acciones condicionda a la validez de una expresión lógica(resto<>0) y no a una cantidad de repeticiones establecida como en el algoritmo E5. Cuando el resto es cero ya se encontró el **mcd** y se termina el algoritmo.

A continuación presentaremos dos ejemplos de resolución de problemas específicos usando el algoritmo EUCLIDES. El procesador que trabaja con el algoritmo es un ser humano adulto que entiende correctamente las acciones y puede ejecutarlas.

Ejemplo para hallar el máximo común divisor de 119 y 544. **Dividendo** almacena 119 y **divisor** almacena 544.

Ejecución 1:

ENTRADA: dividendo ← 119 y **divisor** ← 544 {Ingresa 119 en **dividendo** y 544 en **divisor**}
cociente ← 0 y **resto** ← 119
dividendo← 544 y **divisor** ← 119 {porque **resto** ≠0}
cociente ← 4 y **resto** ← 68
dividendo ← 119 y **divisor** ← 68 {porque **resto** ≠0}
cociente ← 1 y **resto** ← 51
dividendo ← 68 y **divisor** ←51 {porque **resto** ≠0}
cociente ← 1 y **resto** ← 17
dividendo ← 51 y **divisor** ← 17 {porque **resto** ≠0}
cociente ← 3 **resto** ←0
mcd ← 17 {porque **resto** = 0}

SALIDA: mcd ← 17

La ejecución de EUCLIDES parte de los valores **dividendo** ← 119 y **divisor** ← 544 y termina con la

obtención de un resultado (**mcd** ← 17). Se ha resuelto un problema especifico (encontrar el mcd de 119 y 544).

A continuación se presenta otra ejecución del algoritmo (dispondremos de otra entrada a EUCLIDES). Partimos de **dividendo** ← 25 y **divisor** ← 8.

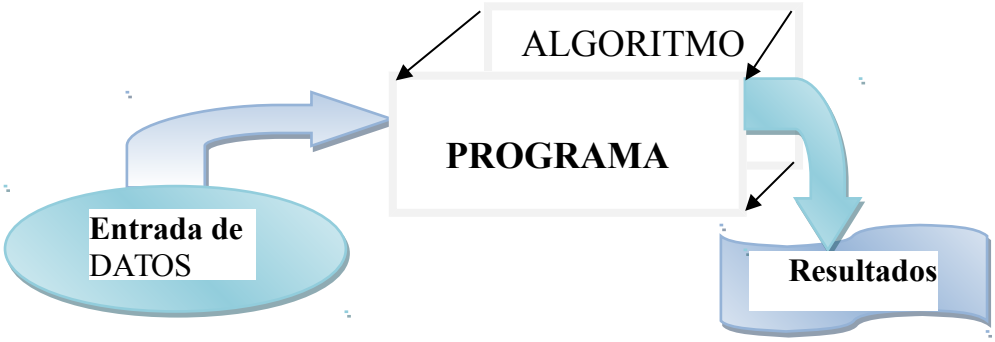
Ejecución 2:
ENTRADA: dividendo ← 25 y **divisor** ← 8
cociente ← 3 y **resto** ← 1
dividendo← 8 y **divisor** ← 1 {porque **resto** ≠0}
cociente ← 8 y **resto** ← 0
mcd ← 1 {porque **resto** = 0}
SALIDA: dcm ← 1

Notemos que el algoritmo EUCLIDES es un método para calcular el mcd de dos enteros positivos cualquiera. En cambio, CASO 1 y CASO 2 son ejecuciones del algoritmo específicas para problemas particulares. En estos casos, el trabajo del algoritmo se inicia con una entrada conteniendo información sobre el problema particular; luego continúa una secuencia de cálculos ejecutados a partir de las acciones primitivas y se termina con la entrega del resultado. CASO 1 y CASO 2 son dos ejemplos de **ejecución** del algoritmo EUCLIDES.
Las entradas se toman sobre conjuntos específicos de objetos. En nuestro ejemplo, los valores de las variables de entrada (**dividendo** y **divisor**) se toman como enteros positivos. Igualmente el resto de las variables (**cociente, resto** y **mcd**).

Debemos tener en cuenta que pueden escribirse diferentes algoritmos para resolver un mismo problema. Una consecuencia a la que nos enfrentamos, entonces, es la elección del mejor algoritmo. Un parámetro para alcanzar una elección acertada es la **eficiencia:** las acciones que se ejecutan deben ser especificadas rigurosamente y sin ambigüedad para cada caso, las operaciones deben ser lo suficientemente básicas para que puedan ser ejecutadas exactamente en un tiempo finito y sin redundancias[1]. Mas tarde volveremos sobre el concepto de eficiencia algorítmica.

2. Programa

Si expresamos un algoritmo en un lenguaje computacional (o también llamado lenguaje de programación), este algoritmo se llama un **programa de computadora**.



[1] El lector interesado puede consultar el libro **Data Structures and Algorithms** (Aho, Hopcroft y Ullman) para ampliar el tema.

El programa (algoritmo traducido a un lenguaje de programación) recibe datos de entrada referidos al problema específico a resolver. El programa los procesa y devuelve resultados con la solución del problema. El proceso es similar al visto recientemente con la ejecución de un algoritmo. En este caso obtenemos la **ejecución de un programa** expresado en un lenguaje de programación, como PASCAL.
Las acciones primitivas del algoritmo se transforman en **sentencias ejecutables** del programa y el procesador que las ejecuta es una **computadora**. La síntesis de la programación tradicional es:

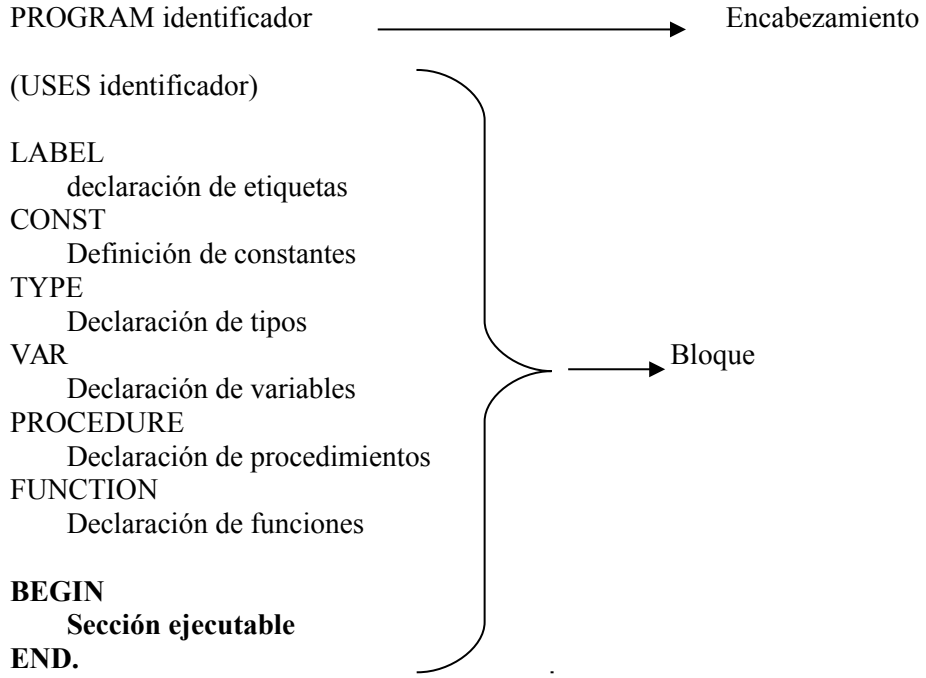
ALGORITMO + DATOS = PROGRAMA

3. Un lenguaje para escribir programas

El lenguaje de programación Pascal fue el primero en incorporarse, en forma coherente, a los conceptos de la programación estructurada definidos por Edsger Dijkstra y C.A.R. Hoare. Pascal fue desarrollado en Zurich por Niklaus Wirth en Eidgenossische Technische Hochschule en 1971[2]; se deriva del lenguaje Algol 60, pero es más completo y fácil de usar. Actualmente, el lenguaje Pascal está ampliamente aceptado como un lenguaje útil que puede ser implementado con eficiencia y como una excelente herramienta de enseñanza.

4. Estructura de un programa Pascal

Un **programa Pascal** consta básicamente de un encabezamiento y un bloque. El encabezamiento está formado por la palabra reservada PROGRAM y un nombre que identifica al programa. Una palabra reservada es propia del lenguaje y tiene un uso específico. El programador no puede usarlas para otros fines. Los identificadores son creados por el programador.
El bloque posee dos secciones principales: una **sección de declaraciones** y **definiciones** (que posee subsecciones iniciadas por palabras reservadas LABEL, CONST, TYPE, VAR, PROCEDURE y FUNCTION) y una **sección ejecutable** encerrada entre BEGIN y END y terminada con un punto (**.**)



[2] N Wirth: "The Programming Language PASCAL". Acta Informatica 1, 35-63, 1971

4.1 Sección de declaraciones

Como vimos en las secciones 1 y 2 de este capítulo, la ejecución del programa necesita la entrada de datos referidos a los problemas específicos a resolver.
El programador Pascal tiene que declarar todas las variables que usará en el programa mediante una **declaración de variable** (entre otras declaraciones, que veremos posteriormente)**.** Declarar una variable implica darle tanto un **nombre (o identificador)** de variable como un **tipo de datos** asociado a esa variable. Por ejemplo, la variable **dividendo** tiene asociado el tipo entero.
En Pascal, un **identificador** sirve para denotar variables, constantes, tipos, procedimientos y funciones y el **tipo de datos** define el conjunto de valores que pueden ser tomados por esa variable y las operaciones que pueden realizarse con ella.

4.1.1 Tipos de datos

Los tipos de datos Pascal se dividen en dos grupos: simples y estructurados. La diferencia es que mientras los primeros se utilizan para almacenar un único valor, los segundos permiten almacenar un conjunto de valores.

- Simples
  - Numérico
    - Entero
    - Real
  - Carácter (char)
  - Lógico (boolean)
  - Cadena de caracteres (String)
- Estructurados
  - Conjuntos (set)
  - Arreglos (array)
  - Registros (record)
  - Archivos (file)

Las palabras que figuran entre paréntesis son palabras reservadas del lenguaje Pascal para los tipos de datos.

4.1.1.1 Datos simples

Los tipos de datos simples reciben también el nombre de estándar, están predefinidos en el lenguaje y disponibles para ser usados directamente por el programador. A continuación se describen sus características y alcances:

- **Entero**: un valor de este tipo es un elemento del subconjunto de los números enteros positivos, negativos y nulo.

Tipo	*Rango de valores que acepta*
Integer	-32,768 a 32,767
Word	0 a 65535
ShortInt	-128 a 127
Byte	0 a 255
LongInt	-2,147,483,648 a 2,147,483,648

- **Real**: un valor de tipo real es un elemento del subconjunto de los números reales, la parte decimal se indica después del punto y es obligatoria.

Tipo	*Rango de valores que acepta*
Real	2.9E-39 a 1.7E38
Single	1.5E-45 a 3.4E38
Double	5.0E-324 a 1.7E308
Extended	1.9E-4851 a 1.1E4932
Comp	-9.2E18 a 9.2E18

- **Boolean**: un valor boolean es uno de los valores lógicos denotados por los identificadores true y false (verdadero y falso respectivamente).

- **Char**: un valor de tipo char, es un elemento del conjunto, finito y ordenado, de caracteres. Todo computador define un conjunto para poder comunicarse, y estos caracteres están disponibles en el equipo de entrada/salida. Cada carácter se expresa entre apóstrofos ''.
  El conjunto debe incluir, entre otros caracteres:
  El conjunto, alfabéticamente ordenado, de las letras mayúsculas y minúsculas latinas:
  'A'..'Z' y 'a'...'z'.
  El conjunto numéricamente ordenado y contiguo, de los dígitos decimales '0'..'9'.
  El carácter de espacio en blanco ' '
  Caracteres especiales: ';', '?', ':', ...

- **String**: es una secuencia de 255 caracteres como máximo, delimitada entre apóstrofos.
  Es posible limitar el número de caracteres de una string especificando entre corchetes dicha cantidad, por ejemplo string [n], con n entero menor a 255. La longitud exacta de la cadena almacenada se obtiene con la función *length(...)*, dicho valor coincide con la posición del último carácter. Es posible el acceso individual a un carácter de la cadena, indicando a continuación del nombre de la variable, su posición entre corchetes.

Los tipos integer, boolean, char se los conoce como ***ordinales***. Un tipo de datos es ordinal porque el conjunto de valores que representa se puede *contar*, es decir, se puede establecer una relación uno a uno entre sus elementos y el conjunto de los números naturales. Esta relación es de orden porque se puede establecer un predecesor y un sucesor para cada valor del tipo ordinal. Esta característica será importante en la implementación de ciertas estructuras de control (Case, For).
Los tipos estructurados se introducen en el Capitulo 6.

4.1.2 Definición de constantes en PASCAL (sección CONST)

Una definición de constante sirve para introducir un identificador como sinónimo de una constante. El valor permanecerá constante a lo largo de la ejecución del programa

Sintaxis:

```
CONST
  Identificador de constante= valor de la constante;
```

El valor de una constante puede ser un número, un caracter o una cadena de caracteres. Posteriormente se analizarán otros casos.
El empleo de los identificadores de constantes mejora la legibilidad del programa y actúa como una ayuda de documentación conveniente. También permite al programador agrupar las cantidades que dependen del

problema al principio del programa donde pueden ser fácilmente localizadas y/o modificadas (mejorando la portabilidad y modularidad del programa).
Ejemplo:

```
CONST
   anio=2005;
   altura=1.68;
   codigo='A5X8';
```

4.1.3 Declaración de variables en PASCAL (sección VAR)

La sección de declaración de variables es VAR. Aquí se escriben los identificadores de las variables y su tipo asociado. Una variable puede cambiar su valor durante la ejecución del programa. Los identificadores deben comenzar con una letra, a continuación puede ir una letra, digito y/o guión bajo) . Observar que la declaración siempre termina con un **;** (punto y coma).

Sintaxis:

VAR Identificador de variable: identificador de tipo;

Ejemplos:

```
VAR
   dividendo, divisor, anio1, anio2:  integer;
   resultado, sueldo_neto: real;
   sigue: boolean;
   nombre: string[25];
```

Observaciones:

- La declaración de variables es obligatoria.
- Cada vez que se declara una variable, se reserva espacio en la memoria para su almacenamiento.

4.1.4 Contador y Acumulador

Un **contador** es una variable de algún tipo entero que cuando se ejecuta se incrementa en una unidad o en una cantidad constante.
Por otra parte, un **acumulador** es una variable que se incrementa en una cantidad variable.

CONTADOR	ACUMULADOR

Begin
Contador:=6;
Contador:=Contador+1;
End;
	Begin
	Acum:=12.32
	S:=7.5;
	Acum:=Acum + S
	End;

4.2 Sección ejecutable

La sección ejecutable contiene a las **sentencias ejecutables** (de aquí en más simplemente **sentencias**). Las sentencias pueden ser simples o estructuradas.
La sentencia es **simple** cuando ninguna de sus partes puede a su vez considerarse como otra sentencia.
Las sentencias **estructuradas** son construcciones compuestas de otras sentencias que deben ser ejecutadas en secuencia (enunciados compuestos), en forma condicional (enunciados condicionales) o en forma reiterada

(enunciados de iteración).
El algoritmo será expresado con estas sentencias (acciones primitivas y no primitivas) y colocado en esta sección.
A continuación veremos las primeras sentencias ejecutables simples en Pascal: asignación y entrada/salida. Notar que el carácter ";" se emplea como separador de sentencias.

4.2.1. Asignación

La acción provocada por el símbolo ← (visto en la sección 1) se traduce en la sentencia de **asignación**. Esto es, sirve para reemplazar el valor actual de una variable por *un nuevo valor* especificado por una expresión. Si no tiene valor, ésta es una forma de inicializarla con alguno. Sintácticamente, el símbolo ← se traduce en **: =** (dos puntos igual)

Sintaxis:

Identificador de variable**:=**expresión;

Por ejemplo estas acciones del algoritmo EUCLIDES
mcd ← divisor
dividendo ← divisor
divisor ← resto
se traducen en Pascal como:
mcd := divisor;
dividendo:= divisor ;
divisor := resto ;

Veamos otros ejemplos:
numero:= 23;
resultado:= 1.2 + 33.45 - **numero**;
resto:= dividendo – cociente * divisor;

Notar que a la derecha del símbolo:= puede figurar una expresión más compleja que una constante o variable; por ejemplo en **resultado** y **resto** se ha almacenado una expresión aritmética. Recordar que a la izquierda de **:=** siempre debe figurar el nombre de una variable.

4.2.2 Entrada / salida

A fin de ingresar los datos a nuestro programa (***entrada***) debemos hacerlo desde un dispositivo (por ejemplo, el teclado) y para mostrar los resultados (***salida***) debemos enviar la información a otro dispositivo (por ejemplo, la pantalla). Las acciones que usaremos son la sentencias PASCAL ***read, readln, write y writeln***.

4.2.2.1 Entrada (lectura de datos)

Si bien el dispositivo de entrada estándar es el teclado, también puede ingresarse datos desde otros dispositivos, como el disco rígido, disco removible, etc. Las sentencias a usar pueden ser ***read*** o ***readln.***

Sintaxis para la entrada estándar:

read(**v1**,**v2**,.....,**vn**); readln(**v1**,**v2**,.....**vn**);

v1,v2,....vn son identificadores de variables indicando las variables donde se almacena el/los valor/es ingresado/s por teclado.

Mientras que ***read*** toma el/los dato/s ingresado/s y el cursor (puntero) queda posicionado en la misma línea***, readln*** cumple la misma función y al detectar el <enter> salta al comienzo de la línea siguiente.

Ejemplo de lectura desde teclado

read(**dividendo, divisor**);
readln(**altura**);

4.2.2.2 Salida (escritura de resultados)

Si bien el dispositivo de salida estándar es la pantalla, también puede escribirse datos en otros dispositivos, como la impresora, el disco rígido, disco removible, etc. Las sentencias a usar pueden ser ***write*** o ***writeln***.

Sintaxis para la salida estándar:

write(**e1,e2,.....,en**); writeln(**e1,e2,.....,en**);

la lista de salida **e1,e2,....,en** son expresiones que indican que valores se escribirán en la pantalla.

Mientras que ***write*** escribe la lista de salida y queda posicionado en la columna siguiente de la misma línea, ***writeln*** salta al comienzo de la línea siguiente.

Ejemplo de escritura en pantalla

```
write(mcd);
write('El dia de ayer fue asueto');
write('La cantidad de mujeres es ', CantM);
writeln('La cantidad de personas es ', CantM + CantH);
```

Escritura con formato

Es posible establecer, para cada elemento de la lista de salida, el ancho del campo donde se escribe. Basta colocar después del elemento el carácter ':' y la cantidad de columnas del campo de escritura.

Writeln(elemento: Ancho);

Ancho es un entero que indica el "ancho del campo" o sea la cantidad de columnas en las que se escribe el elemento. Es utilizado para enteros, cadena, caracteres.

Para los números reales se debe especificar además la cantidad de decimales deseada.

Writeln(elemento real : Ancho: Decimales);

Ancho es un entero que especifica la cantidad de columnas en las que se escribe el elemento real, abarca el total de dígitos del número real (parte entera y decimal), punto decimal y signo.
Decimales es un entero que especifica la cantidad de dígitos decimales con los que se escribirá el número real.

Ejemplos:

A: real; C: integer;

A:= 3.456 ;
C:=10 ;
Write(A: 8:2) ;
Write(C:10) ;
Write(A2:4) ;

S A L I D A

C O L U M N A S

1	2	3	4	5	6	7	8	9	10
				3	.	4	6		
								1	0
3	.	4	5	6	0				

Si no se especifica formato en los números reales, se visualizan en notación científica.

5. Prueba de Escritorio

La prueba de escritorio es usada para probar si un programa tiene errores (no resuelve bien el problema). Esta prueba consiste de una tabla que reproduce la ejecución del programa (en forma similar a la ejecución del algoritmo visto en la sección 1). En la cabecera de esta tabla figuran todos los identificadores de las variables del programa, cada una en una columna (luego veremos otras componentes) y en las sucesivas filas se muestra como se modifica el contenido de esas variables. Una columna a la derecha muestra la salida/s del programa.

Ejemplo:

```
program SuEdad;
 const
       Periodo=5;
 Var
       Nombre : string[25];
       AnioNac, AnioAct:integer;
       Edad:byte;
  begin
        writeln('Ingrese su nombre');
        readln(Nombre);
        writeln('Ingrese el anio de nacimiento y el actual');
        readln(AnioNac, AnioAct);
        Edad:= AnioAct - AnioNac;
        writeln(Nombre, '  tiene ', Edad ,' anios ');
        Edad:= Edad + Periodo;
        writeln('dentro de ', Periodo, ' anios tendra ',Edad);
  end.
```

Prueba de escritorio - Lote de Prueba: Maria, 1995, 2015

Nombre	AnioNac	AnioAct	Edad	Salida
Maria				
	1995			
		2015		
			20	
				Maria tiene 20 anios
			25	
				Dentro de 5 anios tendra 25

6. EJERCICIOS

Consignas de trabajo para los ejercicios 1 y 2
Se detallan distintos lotes de prueba separados por un punto y coma (;), y en cada lote, los distintos valores se separan con comas (,).

1) Escribir las acciones que surgen de la ejecución del algoritmo desarrollado en el ejemplo E4 de la teoría, para los lotes de prueba especificados:
Algoritmo: DECISION_3 Lotes de prueba: 0 , 5 , 8 ; -7 , -8 , 0

2) A partir del programa DISTANCIA, realizar pruebas de escritorio con los lotes de prueba dados, y verificar si se produce algún error:

```
Program DISTANCIA;
 Var
     x1,y1,x2,y2 :integer;
     cuadx,cuady :integer;
     dist : real;
 Begin
      Write('Ingrese coordenadas del punto P: ');
      Readln(x1,y1);
      Write('Ingrese coordenadas del punto Q: ');
      Readln(x2,y2);
      cuadx := SQR(x1-x2);
      cuady := SQR(y1-y2);
      dist := SQRT(cuadx + cuady);
      Writeln('La distancia es ',dist)
 End.
```

Lotes de prueba: 3 , -1 , 4 , 4 **;** 0 , 0 , 8 , 0

3) Editar el siguiente programa:

```
Program Calcular_Area_del_Triangulo;
var
 base,altura:real;
begin
 base:=2.4;
 altura:=5.3;
 writeln(0.5*base*altura);
end.
```

b) Almacenar, en una carpeta identificada con su matrícula en la raíz del disco C, el archivo fuente con el nombre PRUEBA1.
c) Compilar el **archivo fuente**

d) Ejecutar.

e) Agregarle las siguientes líneas luego de **`writeln`**:

```
write('Ingrese base: ');
readln(base);
write('Ingrese altura: ');
readln(altura);
writeln('El area del triángulo con base ', base, ' y altura ' ,
altura, ' es = ', 0.5*base*altura);
```

f) repetir los incisos b) a d) con las modificaciones introducidas grabándolo con el nombre PRUEBA2.

4) Para los programas escritos a continuación se pide:
-Realizar las pruebas de escritorio correspondientes con los lotes de prueba dados.
-Transcribir los programas en el Editor de Freepascal o Geany y ejecutarlos con distintos lotes de prueba.
-Agregar las leyendas aclaratorias que considere necesarias para lograr una buena comunicación con el usuario.
Observación: Para realizar los ejercicios que se presentan a continuación, deberá consultar los temas Escritura con Formato y la sentencia Uses.

a)
```
Program UNO;
const
   a=3;
var
   x,y,z:integer;
   T:real;
begin
   readln(x);
   readln(y);
   y:=a*x;
   y:=y+a;
   writeln(y);
   x:=y;.
   y:=a;
   writeln(a, ' ',x, ' ',y)
   T:=(x+y)/(x-y);
   writeln('El resultado es ',T:7:2);
   z:=(x+y)div(x-y);
   writeln('El resultado es ',z:7);
   z:=(x+y)mod(x-y);
   writeln('El resultado es ',z:7);
end.
```

Lote de prueba: 4 , 5

b)
```
Program TRES;
   var
   c: char;
begin
   c:= 'T';
   writeln ('Esta es la letra c');
   writeln('Esta es la letra ',c);
end.
```

c)
```
Program CUATRO;
const
   x1=10;
var
   imp,expon,val1,val2,tot:integer;
   medi:real;
```

```
begin
   imp:=100;
   expon:=20;
   val1:=imp-expon;
   val2:=x1;
   tot:=val1+val2;
   medi:=tot*50+10/val1;
   writeln('imp':10, 'expon':10, 'val1':10, 'val2':10, 'total':10, 'medi':10);
   writeln;
   writeln(imp:10,expon:10,val1:10,val2:10,tot:10,medi:10:2)
end.
```

5) Analizar los tipos de datos en Pascal y completar las siguientes tablas:

a)Tipo de Dato	Valor Mínimo	Valor Máximo	Bytes que ocupa en memoria
Shortint			
Integer			
Longint			
Byte			
Word			
Cardinal			
Real			
Single			
Double			
Extended			

b) TIPO	Ejemplos de valores que puede tomar
Char	
String	
Boolean	

6) En la columna de la izquierda se indican posibles datos que corresponderían a problemas a ser resueltos mediante un programa Pascal. Indique con una cruz cuál es el tipo apropiado con el que debe declararse la variable en la que se almacenen dichos datos, teniendo en cuenta la adecuación entre los posibles valores y el tipo de datos, como así también el costo de almacenamiento en memoria.

DATOS DEL PROBLEMA	**Integer**	**Word**	**Byte**	**Shortint**	**Longint**	**Cardinal**	**Real**	**Double**	**String**
Estatura de una persona en metros									
Estatura de una persona en centímetros enteros									
Resultados de un censo, en cantidades									
Resultados de un censo, en porcentajes									
Magnitudes referidas a distancias en km correspondientes a datos astronómicos									
Magnitudes referidas a kilometrajes mensuales de una empresa de transportes de larga distancia									
Magnitudes referidas a kilometrajes diarios de una empresa de transportes urbanos									
Cantidad de televisores marca A vendidos anualmente por la fábrica en todo el país									
Cantidad de televisores marca A vendidos mensualmente por un comercio en Mar del Plata									
Número de DNI de una persona									
Número de la casa o edificio de un domicilio.									

7) Editar y ejecutar los siguientes programas, comparar sus salidas y fundamentar las diferencias encontradas:

```
Program tipos;
var
   p,a,b:byte;
begin
   a:=250;
   p:=a+10;
   writeln('El resultado es ',p);
   b:=4;
   p:=b-8;
   writeln('El valor de b es ',p);
end.
```

```
Program tipos2;
var
   a,b:byte;
   p:integer;
begin
   a:=250;
   p:=a+10;
   writeln('El resultado es ',p);
   b:=4;
   p:=b-8;
   writeln('El valor de b es ',p);
end.
```

8) Investigar sobre el código ASCII (American Standard Code for Information Interchange) y responder las siguientes preguntas:
¿Cuáles son los códigos numéricos correspondientes a los caracteres `'@'` y `' '` espacio?
¿Cuál es la diferencia entre los códigos numéricos de una letra minúscula y su mayúscula?

9)

a) Ejecutar 2 veces el siguiente programa ingresando, respectivamente, los valores 7 y 15.

```
{$R+} {Directiva de compilador R}
Program Positivos;
Type
 NumPositivo = 0..10;
Var
 numero: NumPositivo;
Begin
 Write('Escribe un número entero positivo : ');
 Readln(numero);
end.
```

b) Repetir las ejecuciones solicitadas en el inciso a) pero eliminando previamente del programa la directiva de compilación escribiendo {$R-}

Nota: Pascal no siempre produce un error cuando el valor de un tipo subrango está fuera de su rango definido. Sin embargo se puede tener la posibilidad de visualizar dichos errores mediante la directiva de compilador:{$R+} activada y {$R-} desactivada. Por defecto está desactivada.

10) Escribir un programa Pascal que lea una palabra en una variable String. El programa debe informar por pantalla: cuántas letras tiene la palabra, su primera letra y su última letra.
Nota: se sugiere releer la teoría asociada a cadena de caracteres.